

```

ORG 100h

; Hardcoded 4-digit password: 1 2 3 4
PASSWORD DB 1,2,3,4
MAX_TRY   DB 4

; Seven-segment lookup table (common cathode)
SEGMENT_TABLE DB 3Fh, 06h, 5Bh, 4Fh, 66h, 6Dh, 7Dh, 07h, 7Fh, 6Fh

; Data to store user input
USER_INPUT DB 4 DUP(0)
WRONG_COUNT DB 0

;-----
; Delay routine (simple)
DELAY:
    MOV CX, 0FFFFh
D_LOOP:
    LOOP D_LOOP
    RET
;-----

;-----
; Read key from 3x3 keypad
; Returns AL = digit 1..9
READ_KEY:
    IN AL, A0h      ; Read keypad port
    AND AL, 0FH     ; Lower 4 bits for simplicity
    ; Map raw value to 1..9
    ; Assuming simple mapping: 1->1, 2->2,...9->9
    ; Adjust according to your hardware scan
    CMP AL, 0
    JE READ_KEY     ; No key pressed, loop
    RET
;-----

;-----
; Main routine
MAIN:
    MOV BL, 0       ; Wrong attempt counter

TRY_LOOP:
    MOV SI, 0       ; Index for user input

INPUT_LOOP:
    CALL READ_KEY
    MOV USER_INPUT[SI], AL
    INC SI
    CMP SI, 4
    JL INPUT_LOOP   ; Read 4 digits

    ; Compare entered password with PASSWORD

```

```

    MOV SI, 0
    MOV DI, 0
CHECK_LOOP:
    MOV AL, USER_INPUT[SI]
    CMP AL, PASSWORD[SI]
    JNE WRONG_PASS
    INC SI
    CMP SI, 4
    JL CHECK_LOOP

    ; Password correct
    JMP PASS_OK

WRONG_PASS:
    INC BL                      ; Increment wrong attempt counter
    MOV AL, BL
    MOV AH, 0
    ; Display wrong count on 7-seg
    MOV AL, SEGMENT_TABLE[AL-1]
    OUT 90h, AL

    CMP BL, 4
    JL TRY_LOOP                ; Less than 4, retry

    ; 4 wrong attempts -> blink LED
    MOV CX, 8                  ; Blink 8 times
BLINK_LOOP:
    MOV AL, 0FFh
    OUT 80h, AL
    CALL DELAY
    MOV AL, 00h
    OUT 80h, AL
    CALL DELAY
    LOOP BLINK_LOOP

    JMP MAIN                    ; Reset system

PASS_OK:
    ; Password accepted
    ; Turn on LED steady
    MOV AL, 0FFh
    OUT 80h, AL
    ; Optional: Turn on specific 7-seg code or leave as is
    JMP MAIN

```

```

#include <stdio.h>
#include <stdint.h>

#define ROWS 4
#define COLS 3

// Example hardware port reading/writing functions
uint8_t read_columns();
void write_row(uint8_t pattern);

uint8_t ROW_TABLE[ROWS] = {0x0E, 0x0D, 0x0B, 0x07}; // One row
LOW at a time
char KEY_TABLE[ROWS][COLS] = {
    {'1', '2', '3'},
    {'4', '5', '6'},
    {'7', '8', '9'},
    {'*', '0', '#'}
};

char read_key() {
    for (int row = 0; row < ROWS; row++) {
        write_row(ROW_TABLE[row]); // Activate row by out command
        uint8_t col_val = read_columns() & 0x07; // Read lower 3
bits
        if (col_val != 0x07) { // 111b means no key
            // Determine which column is LOW
            int col = -1;
            if ((col_val & 0x01) == 0) col = 0;
            else if ((col_val & 0x02) == 0) col = 1;
            else if ((col_val & 0x04) == 0) col = 2;
            if (col != -1)
                return KEY_TABLE[row][col]; // Return key
        }
    }
    return 0; // No key pressed
}

```

```

PORT_ROW EQU 90h
PORT_COL EQU 91h

ROW_TABLE DB 0Eh,0Dh,0Bh,07h
KEY_TABLE DB '1','2','3','4','5','6','7','8','9','*','0','#'

DELAY:
    MOV CX,2000
D_LOOP: LOOP D_LOOP
    RET

SCAN_KEY:
    MOV CX,4                ; 4 rows
    MOV SI, OFFSET ROW_TABLE ; pointer to ROW_TABLE
    MOV DI, OFFSET KEY_TABLE ; pointer to KEY_TABLE

NEXT_ROW:
    MOV AL, [SI]            ; AL = ROW_TABLE[row]
    OUT PORT_ROW, AL
    CALL DELAY

    IN AL, PORT_COL
    AND AL, 07h             ; mask 3 columns
    CMP AL, 07h
    JE NO_KEY_IN_ROW

    MOV BL, AL
    NOT BL
    AND BL, 07h

    ; Determine column index
    MOV AL, BL
    CMP AL, 1
    JE COL0
    CMP AL, 2
    JE COL1
    CMP AL, 4
    JE COL2
    JMP NO_KEY_IN_ROW

COL0: MOV BX,0
    JMP FOUND
COL1: MOV BX,1
    JMP FOUND
COL2: MOV BX,2

FOUND:
    MOV DX,4
    SUB DX,CX                ; row index = 4 - CX
    MOV AX, DX
    MOV BP,3
    MUL BP                   ; AX = row*3

```

```
ADD AX, BX      ; AX = row*3 + col
MOV AL, [DI+AX] ; get key from table
RET
```

```
NO_KEY_IN_ROW:
  INC SI      ; move to next row pattern
  LOOP NEXT_ROW
  MOV AL, 0
  RET
```

Sure! Let's make a **general solution for controlling a stepper motor**. I'll provide it **step by step**:

- **Step 1:** Algorithm in English
- **Step 2:** C code
- **Step 3:** 8088 ASM code

We'll assume:

- A **4-wire stepper motor**
 - Connected to **ports 80h–83h** (or one port for simplicity, e.g., 80h)
 - **Full-step drive**
 - Direction control via sequence of outputs
-

Algorithm (English)

Goal: Rotate stepper motor by N steps in a chosen direction.

1. **Initialize**

- Set motor control pins (output port)
- Define **step sequence** (4 steps for full-step, 8 for half-step)

2. **Choose rotation direction**

- Forward → sequence from step1 → stepN
- Reverse → sequence from stepN → step1

3. **Loop for number of steps**

- Output current step pattern to motor pins
- Wait small delay for motor to move
- Move to next step in sequence

4. **Repeat until all steps are done**

C Code

```
#include <stdio.h>
#include <stdint.h>
#include <dos.h> // For outportb/delay (Turbo C/8088 DOS)

#define MOTOR_PORT 0x80

// Full-step sequence for 4-wire stepper motor
```

```

uint8_t step_seq[4] = {0x09, 0x0C, 0x06, 0x03}; // Example: coil A+B, B+C, C+D,
D+A

void delay_ms(int t) {
    int i,j;
    for(i=0;i<t;i++)
        for(j=0;j<1000;j++);
}

void step_motor(int steps, int direction) {
    int i, index = 0;
    for(i=0;i<steps;i++) {
        outportb(MOTOR_PORT, step_seq[index]); // output step pattern
        delay_ms(10);                          // small delay
        if(direction == 1) {                    // forward
            index = (index + 1) % 4;
        } else {                                // reverse
            index = (index + 3) % 4;             // -1 modulo 4
        }
    }
}

int main() {
    step_motor(100,1); // rotate 100 steps forward
    step_motor(50,0);  // rotate 50 steps reverse
    return 0;
}

```

Explanation:

- `step_seq` → defines output pattern for motor coils
 - `step_motor()` → loops through steps, outputs sequence, adds delay
 - `direction = 1` → forward, `0` → reverse
-

ASM Code (8088)

```

MOTOR_PORT EQU 80h

STEP_SEQ DB 09h,0Ch,06h,03h ; Full-step 4-wire

; Simple delay
DELAY:
    MOV CX,2000
D_LOOP: LOOP D_LOOP
    RET

;-----
; STEPPER_MOTOR
; AX = steps
; BL = direction (1=forward,0=reverse)
;-----
STEPPER_MOTOR:
    MOV SI,0 ; step index
    MOV CX,AX ; number of steps

NEXT_STEP:
    MOV AL, STEP_SEQ[SI]

```

```

    OUT MOTOR_PORT, AL
    CALL DELAY

    CMP BL,1
    JE FORWARD
    ; reverse
    SUB SI,1
    CMP SI,0FFh      ; if negative
    JAE OK_REVERSE
    MOV SI,03h
OK_REVERSE:
    JMP STEP_DONE

FORWARD:
    INC SI
    CMP SI,04h
    JL STEP_DONE
    MOV SI,0

STEP_DONE:
    LOOP NEXT_STEP
    RET

```

Explanation:

- STEP_SEQ → same as step_seq in C
- SI → keeps **current step index**
- CX → number of steps (like loop counter)
- BL → direction
- Forward → increment index, wrap around at 4
- Reverse → decrement index, wrap around at 0xFF → 3
- OUT MOTOR_PORT, AL → sends output to motor
- DELAY → small wait for motor to step

✅ **This is a complete, general solution** for rotating a stepper motor forward/reverse using both C and ASM.

I can also **make it accept different rotation speeds** by changing delay dynamically, like in a more advanced motor control program.

Do you want me to add that feature?